PROJECT CODE: TMA-2026

TASK MANAGEMENT WEB APPLICATION

Architecture & Design

DOCUMENT DETAILS

Version: v1.0

Date: March 14, 2026

PROJECT INFORMATION

Prepared by: Architecture Team

Methodology: Waterfall

PROJECT OVERVIEW: BUILDING A SCALABLE TASK MANAGEMENT PLATFORM

Context & Objectives

The Task Management Web Application is designed as an internal collaboration tool for small organizations with up to 50 users.

The architecture prioritizes security, maintainability, and independent scalability of each system tier, enabling efficient task management with role-based authorization.

Key Parameters

CURRENT PHASE
Phase 3 (Design)

TARGET AUDIENCE
Internal Teams (≤50 users)

GO-LIVE TARGET
Q3 2026

CLEARLY DEFINED SCOPE ENSURES FOCUSED DEVELOPMENT

In Scope

- User authentication, authorization, and role management
- Complete task lifecycle: creation, assignment, prioritization, and status tracking
- Dashboard with summary metrics and activity feeds
- Email and in-app notifications for task events
- REST API backend with JSON responses
- Responsive single-page front-end application
- Relational database persistence with Redis session caching

Out of Scope (Future Phases)

- Mobile native applications (iOS/Android)
- Third-party calendar integrations
- AI-powered task suggestions
- Multi-tenancy and SaaS features

THREE-TIER ARCHITECTURE ENABLES INDEPENDENT SCALING

The app adopts a three-tier client-server architecture separating presentation, logic, and data. This modular design enables independent deployment and simpler development.

Client Tier

React 18 SPA in the browser.

- REST over HTTPS
- Local state & UI
- Decoupled from backend

Application Tier

Node.js + Express API server.

- Business logic
- Auth & validation
- Service orchestration

Data Tier

PostgreSQL 16 for persistence.

- ACID compliance
- Redis caching
- Independent schema

MODERN TECHNOLOGY STACK BALANCES MATURITY WITH DEVELOPER PRODUCTIVITY

Frontend Layer

React 18.3
Mature ecosystem with extensive component reuse capabilities

Tailwind CSS 3.4
Utility-first styling for rapid UI iteration

Zustand 4.x
Lightweight state management without boilerplate

Vite 5.x
Fast build tool with optimized bundle output and HMR

Backend Layer

Node.js 20 LTS
Non-blocking I/O with full-stack JavaScript capability

Express 4.18
Minimal, well-understood web framework with extensive middleware ecosystem

Sequelize 6.x
ORM providing migration support and PostgreSQL compatibility

Data & Infrastructure

PostgreSQL 16
ACID-compliant relational database with rich SQL capabilities

Redis 7
Sub-millisecond latency for session storage and pub/sub messaging

Docker 25 & Compose 2.x
Reproducible environments for local development and CI/CD

GitHub Actions
Native SCM integration for automated testing and deployment

AWS (EC2, RDS, ElastiCache)
Cost-effective cloud infrastructure for small-scale deployment

SINGLE-PAGE APPLICATION DELIVERS RESPONSIVE UX

The frontend is a React 18 SPA communicating with the backend via REST APIs. Key frontend components include:

Router

React Router v6 for SPA navigation and protected routes.

Auth Module

Custom hooks with JWT handling; tokens in httpOnly cookies.

Task Module

Zustand-based components for CRUD and Kanban lists.

Dashboard

KPI cards and charts for quick insights.

Notification UI

Real-time alerts via WebSocket with dismissable toasts.

API Client

Axios interceptors for centralized HTTP and token handling.

Compiled static assets are served via Nginx for caching and CDN distribution. Tailwind CSS accelerates responsive styling.

MODULAR BACKEND SERVICES PROVIDE CLEAR SEPARATION OF LOGIC

Auth Service

Issues and validates JWT tokens, manages refresh token rotation, implements account lockout, and enforces bcrypt password hashing.

Task Service

Manages the complete task lifecycle (creation, assignment, updates, deletion, filtering) using Sequelize ORM for database abstraction.

User Service

Handles user profiles, role assignment (admin, manager, member), and team membership management.

Notification Service

Sends email alerts on task events and queues in-app notifications asynchronously using Node-cron and Nodemailer.

Middleware Stack

Helmet.js for security headers, CORS policy enforcement, and per-IP rate limiting (100 req/min standard).

Logging

Winston and Morgan provide structured JSON logs and comprehensive request/response audit trails for troubleshooting.

All services run within a single Express application on Node.js 20 LTS, enabling simplified deployment while maintaining logical separation of concerns through clear module boundaries.

POSTGRESQL SCHEMA SUPPORTS COMPLETE TASK LIFECYCLE

The Task Management Web Application uses PostgreSQL 16 as its primary data store, implementing a relational schema with five core entities.

Users Table

- UUID, email (unique), bcrypt-hashed password, full name, role (admin/manager/member)
- Supports soft-delete via is_active flag

Projects Table

- UUID, name, description, owner reference, status (active/archived/completed)
- Enables project-level organization and access control

Tasks Table

- Title, description, project reference, assignee, reporter, due date
- Status (todo/in_progress/review/done), priority (low/medium/high/critical)
- Supports filtering, sorting, and comprehensive lifecycle tracking

Comments & Notifications

- **Comments:** Task comments for collaboration and discussion threads
- **Notifications:** Audit trail of notification events sent to users

Audit & Security: All tables include `created_at` and `updated_at` timestamps managed by Sequelize hooks for comprehensive audit trails. The schema uses UUID primary keys for security and distributed system compatibility.

STANDARDIZED REST API ENSURES PREDICTABLE INTEGRATIONS

API Conventions

- **Base URL:**
  `https://api.tma.internal/v1`
- **Content-Type:**
  `application/json` for all requests and responses
- **Authentication:**
  Bearer JWT in Authorization header (except /auth endpoints)
- **Error Responses:**
  Follow RFC 7807 Problem Details format
- **Pagination:**
  `?page=1&limit=20` query parameters on list endpoints

Key Endpoints

Authentication

```
POST   /auth/register, /auth/login, /auth/refresh
POST   /auth/logout, /auth/forgot-password
PATCH  /auth/reset-password
```

Tasks

```
GET    /tasks (with filters), /tasks/:id
POST   /tasks, /tasks/:id/comments
PUT    /tasks/:id | PATCH  /tasks/:id | DEL    /tasks/:id
```

Users & Projects

```
GET    /users, /users/:id, /projects
POST   /projects
PATCH  /users/:id | PUT    /projects/:id | DEL    /projects/:id
```

Each endpoint enforces appropriate authentication and authorization checks, with role-based access control determining which users can perform which operations.

JWT-BASED AUTHENTICATION PROVIDES STATELESS, SCALABLE SECURITY

The Task Management Web Application implements a sophisticated JWT-based authentication system that balances security with scalability.

Token Strategy

- **Access tokens** expire after 15 minutes, forcing frequent rotation and limiting exposure window.
- **Refresh tokens** expire after 7 days and are stored server-side in Redis.
- Refresh tokens are **invalidated on logout**, preventing token reuse after explicit user sign-out.

Password Security

- All passwords are hashed using **bcrypt with cost factor 12**, providing strong resistance against brute-force attacks.
- **Account lockout** is enforced after 5 consecutive failed login attempts.
- Lockout includes a **30-minute block period** preventing automated attacks.

Authorization Model

- **Role-Based Access Control (RBAC)** with three tiers: admin > manager > member.
- **Admin:** Manage all system resources and users.
- **Manager:** Create and manage projects.
- **Member:** View and work on assigned tasks.
- All authorization checks are enforced at the **API endpoint level**.

MULTI-LAYERED SECURITY STRATEGY PROTECTS DATA AND APIS

The Task Management Web Application implements a comprehensive defense-in-depth security strategy across all layers to ensure robust protection against modern threats.

Transport Security

- **HTTPS Enforcement:** Via CloudFront + AWS Certificate Manager; all HTTP traffic redirected to HTTPS.
- **HSTS Header:** Configured with max-age=31536000 and includeSubDomains, forcing secure connections for one year.
- **Security Headers:** Helmet.js configures X-Frame-Options (DENY), X-Content-Type-Options (nosniff), and Content Security Policy.

API Security

- **CORS Policy:** Restricted to explicitly listed origins only; credentials: true for cookie support.
- **Rate Limiting:** 100 requests/min per IP (Redis-backed); stricter 10 req/min on /auth endpoints.
- **Input Validation:** Joi schema validation on all POST/PUT/PATCH endpoints prevents injection attacks.
- **SQL Injection Prevention:** Enforced by Sequelize parameterized queries.

Data Security

- **Network Isolation:** All PII stored in PostgreSQL within private VPC subnets (not publicly accessible).
- **Encryption at Rest:** Implemented via AWS RDS storage encryption (AES-256).
- **Secrets Management:** Environment secrets managed via AWS Secrets Manager; never committed to source control.
- **Disaster Recovery:** Database backups are encrypted and retained securely.

CLOUD-NATIVE DEPLOYMENT ENSURES HIGH AVAILABILITY

The Task Management Web Application is deployed on AWS infrastructure in the ap-southeast-1 region, implementing a multi-availability zone architecture for high availability.

Infrastructure Components

- **Route53:** DNS service providing domain name resolution and health checks.
- **CloudFront:** CDN and WAF providing DDoS protection, edge caching, and SSL/TLS termination.
- **VPC (10.0.0.0/16):** Isolated network with public and private subnets.
- **EC2 Instances:** Two t3.small instances running the Node.js API and React SPA (behind ALB).
- **RDS PostgreSQL:** db.t3.medium instance in private subnet with automated backups and Multi-AZ failover.
- **ElastiCache Redis:** Session storage and JWT blocklist with automatic failover.

Environment Strategy

- **Development:**
  Local Docker Compose environment for developer machines.
- **Staging:**
  AWS infrastructure with anonymized production data clone for UAT and integration testing.
- **Production:**
  Dual EC2 instances behind Application Load Balancer, with RDS Multi-AZ and ElastiCache failover.

AUTOMATED CI/CD PIPELINE ENSURES QUALITY AND RAPID ITERATION

The Task Management Web Application implements a comprehensive CI/CD pipeline using GitHub Actions, automating testing, building, and deployment.

01 Feature Branch

Push to feature branch triggers automated lint checks and unit tests; failures block merge.

02 Pull Request

Code review required; all tests must pass before merge approval.

03 Merge to Main

Automatically builds Docker image and pushes to Amazon ECR.

04 Staging Deployment

Automatic deployment to staging environment for integration testing.

05 Production Deployment

Manual approval gate required before production deployment.

06 Database Migrations

Sequelize CLI migrations run as pre-deploy step with automatic rollback on failure.

07 Rollback

Previous ECR image tag can be re-deployed via GitHub Actions workflow_dispatch for rapid incident response.

This pipeline ensures that all code changes are tested, built, and deployed consistently, reducing manual errors and enabling rapid iteration while maintaining production stability.

PROACTIVE RISK MANAGEMENT ENSURES PROJECT SUCCESS

The architecture design identifies and addresses key technical risks and establishes foundational assumptions.

High-Impact Risks

- **DB Migration Failures:** Automated migration tests in staging and prepared rollback scripts mitigate production impact.
- **EC2 Single Point of Failure:** Application Load Balancer with two EC2 instances in separate availability zones provides automatic failover.
- **Over-Permissive CORS:** Strict origin whitelist enforced and reviewed per release.

Medium-Impact Risks

- **Redis Unavailability:** Graceful degradation to database-based session fallback maintains functionality.
- **JWT Secret Rotation:** Key ID (kid) claim implementation enables zero-downtime rotation.
- **npm Vulnerabilities:** Dependabot alerts and weekly npm audit in CI catch vulnerabilities early.

Key Assumptions

- Maximum concurrent users will not exceed 50 during initial rollout.
- AWS ap-southeast-1 region is acceptable for data residency requirements.
- SMTP relay service (AWS SES) is available for email notifications.
- All team members have GitHub and AWS console access.
- Waterfall phase gates will be formally signed off before proceeding.

WELL-DEFINED DATA FLOWS ENSURE PREDICTABLE SYSTEM BEHAVIOR

The Task Management Web Application implements two primary data flows that exemplify system operation:

Authentication Flow

1. User submits credentials via Login form (`HTTPS POST /auth/login`).
2. Auth Service validates credentials against bcrypt-hashed password in PostgreSQL.
3. On success: JWT access token (15 min expiry) + refresh token (7 days) returned to client.
4. Refresh token stored server-side in Redis; access token stored client-side in memory.
5. All subsequent API calls include `Authorization: Bearer <token>` header.
6. Middleware validates JWT signature and expiry before processing request.

Task Retrieval Flow

1. Authenticated user sends `GET /tasks` request with optional filters (project, status, priority, assignee).
2. Middleware validates JWT signature and expiry.
3. Task Service executes Sequelize ORM query against PostgreSQL with filters applied.
4. Response payload JSON-serialized and returned with HTTP 200.
5. Notification Service triggered asynchronously on task assignment or status change.
6. Email and in-app notifications queued for affected users.

These flows ensure that all operations are authenticated, authorized, and properly logged for audit compliance.

ROBUST ARCHITECTURE PROVIDES FOUNDATION FOR SCALABLE TASK MANAGEMENT

The Task Management Web Application architecture delivers several critical strengths:

Technical Excellence

- Three-tier architecture enables independent scaling and maintenance of each component.
- Modern, production-ready technology stack balances maturity with developer productivity.
- Comprehensive REST API with standardized conventions enables predictable integrations.
- Sophisticated JWT-based authentication provides stateless, scalable security.

Operational Resilience

- Multi-AZ AWS deployment with automatic failover ensures high availability.
- Automated CI/CD pipeline reduces manual errors and enables rapid iteration.
- Comprehensive logging and audit trails support troubleshooting and compliance.
- Graceful degradation strategies ensure continued operation during component failures.

Security & Compliance

- Defense-in-depth security strategy protects data across all layers.
- RBAC authorization model ensures users can only access appropriate resources.
- Encryption at rest and in transit protects sensitive data.
- Comprehensive risk management identifies and mitigates technical threats.

Scalability & Maintainability

- Modular backend services enable independent scaling and development.
- Database schema supports complete task lifecycle with audit trails.
- Clear separation of concerns simplifies debugging and reduces technical debt.
- Waterfall methodology with formal phase gates ensures quality and stakeholder alignment.

CLEAR PATH FORWARD: FROM DESIGN TO IMPLEMENTATION

The Task Management Web Application is currently in Phase 3 (Design), with the following planned milestones:

Immediate Next Steps

- **Formal sign-off** of this High-Level Design by all stakeholders (Lead Architect, PM, CTO, QA Lead).
- **Transition to Phase 4:** Low-Level Design (LLD) development.
- Detailed component-level design specifications for each backend service.
- Frontend component library and interaction patterns documentation.

Upcoming Deliverables

- **Low-Level Design (LLD)** document with detailed implementation specifications.
- **Comprehensive Test Plan** covering unit, integration, and UAT testing.
- **Deployment Runbook** with operational procedures and incident response playbooks.
- **Security assessment** and penetration testing results.

Timeline

- **Low-Level Design completion**
  April 2026
- **Development phase**
  May-June 2026
- **Testing and UAT**
  July 2026
- **Production deployment**
  Q3 2026 (Go-Live target)

This phased approach ensures thorough planning and quality execution, reducing implementation risks and enabling successful delivery.